

Stochastic Gradient Descent (SGD)

Mathematical Foundations, PyTorch Implementation & Chain Rule Derivations

1 PyTorch Implementation

Below is a complete implementation of a 2-layer fully connected network trained with Stochastic Gradient Descent (SGD) on individual training samples in PyTorch.

```
import torch
import torch.nn as nn
import torch.optim as optim

class TwoLayerNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(TwoLayerNet, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim) # Layer 1: W1, b1
        self.sigmoid = nn.Sigmoid() # Activation Function
        self.fc2 = nn.Linear(hidden_dim, output_dim) # Layer 2: W2, b2

    def forward(self, x):
        z1 = self.fc1(x)
        a1 = self.sigmoid(z1)
        z2 = self.fc2(a1)
        y_hat = z2 # Linear output layer
        return y_hat, a1, z1

# Hyperparameters & Model Setup
input_dim, hidden_dim, output_dim = 3, 4, 1
lr = 0.01

model = TwoLayerNet(input_dim, hidden_dim, output_dim)
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)

# Dummy single sample (Stochastic step: batch size = 1)
x = torch.randn(1, input_dim) # Shape: (1, 3)
y = torch.randn(1, output_dim) # Shape: (1, 1)

# Forward pass, Loss calculation, Backward pass, Weight Update
y_hat, a1, z1 = model(x)
loss = criterion(y_hat, y)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

2 Forward Pass Mathematics

Consider a single input instance $x \in \mathbb{R}^{1 \times d_{in}}$.

Layer 1 Computation

Let $W_1 \in \mathbb{R}^{d_{in} \times d_{hidden}}$ and bias $b_1 \in \mathbb{R}^{1 \times d_{hidden}}$:

PRE-ACTIVATION 1

$$z_1 = x W_1 + b_1$$

Applying the element-wise Sigmoid activation function $\sigma(z) = 1 / (1 + e^{-z})$:

ACTIVATION 1

$$a_1 = \sigma(z_1)$$

Layer 2 Computation

Let $W_2 \in \mathbb{R}^{d_{hidden} \times d_{out}}$ and bias $b_2 \in \mathbb{R}^{1 \times d_{out}}$:

OUTPUT PREDICTION

$$\hat{y} = z_2 = a_1 W_2 + b_2$$

3 Loss Function (Mean Squared Error)

For a single output prediction $\hat{y}$ and target y , the Mean Squared Error (MSE) loss L is:

MSE LOSS FORMULA

$$L = \frac{1}{2} \| \hat{y} - y \|_2^2 = \frac{1}{2} \sum_{k=1}^{d_{out}} (\hat{y}_k - y_k)^2$$

Note: The scaling factor of $\frac{1}{2}$ simplifies derivative calculations during backpropagation.

4 Gradient Computation via Chain Rule

Layer 2 Gradients

Derivative of loss with respect to output prediction $\hat{y}$:

$$\partial L / \partial \hat{y} = (\hat{y} - y)$$

Applying chain rule for parameter weights W_2 and bias b_2 :

$$\partial L / \partial W_2 = a_1^T (\hat{y} - y)$$

$$\partial L / \partial b_2 = (\hat{y} - y)$$

Layer 1 Gradients

Backpropagating error into hidden layer activations a_1 :

$$\partial L / \partial a_1 = (\hat{y} - y) W_2^T$$

Propagating error through the Sigmoid activation derivative where $\sigma'(z) = \sigma(z) \odot (1 - \sigma(z))$:

ERROR TERM (Δ_1)

$$\delta_1 = \partial L / \partial z_1 = [(\hat{y} - y) W_2^T] \odot [a_1 \odot (1 - a_1)]$$

Gradients for Layer 1 weights W_1 and bias b_1 :

LAYER 1 GRADIENTS

$$\partial L / \partial W_1 = x^T \delta_1$$

$$\partial L / \partial b_1 = \delta_1$$

5 Weight Update Formula (SGD)

Parameters are updated immediately after processing each training sample using learning rate η :

LAYER 2 PARAMETERS

$$W_2^{(t+1)} = W_2^{(t)} - \eta (\partial L / \partial W_2)$$

$$b_2^{(t+1)} = b_2^{(t)} - \eta (\partial L / \partial b_2)$$

LAYER 1 PARAMETERS

$$W_1^{(t+1)} = W_1^{(t)} - \eta (\partial L / \partial W_1)$$

$$b_1^{(t+1)} = b_1^{(t)} - \eta (\partial L / \partial b_1)$$